

Chapter 3

Concurrency Control Techniques

Concurrency Control

- is the process of managing simultaneous operations on the database without having them interfere with one another.
- Prevents interference when two or more users are accessing database simultaneously and at least one is updating data.
- Although two transactions may be correct in themselves, interleaving of operations may produce an **incorrect result**.
- Three basic concurrency control techniques:
 - **Locking methods**
 - **Time stamping**
 - **Optimistic**

...CON'T

- **Locking** and **Time stamping** are pessimistic approaches since they delay transactions.
- Both Locking and Time stamping are conservative approaches:
 - delay transactions in case they conflict with other transactions.
- The optimistic approach allows us to proceed and check conflicts at the end.
- Optimistic methods assume conflict is *rare* and only check for conflicts at commit.

Locking Method

- The locking method is a mechanism for preventing simultaneous access on a shared resource for a critical operation
- A **LOCK** is a mechanism for enforcing limits on access to a resource in an environment where there are many threads of execution.
- Locks are one way of enforcing concurrency control policies.
- Transaction uses locks to deny access to other transactions and prevent **incorrect** updates.
- Lock prevents another transaction from modifying item or even reading it, in the case of a write lock.
- **Lock (X)**: If a transaction T_1 applies Lock on data item X, then X is locked and it is not available to any other transaction.
- **Unlock (X)**: T_1 Unlocks X. X is available to other transactions.

Types of a Lock

- **Shared lock:** A Read operation does not change the value of a data item.
 - Hence a data item can be read by two different transactions simultaneously under share lock mode.
 - So only to read a data item T_1 will do:
 - *Share lock (X), then Read (X), and finally Unlock (X).*
- **Exclusive lock:** A write operation changes the value of the data item.
 - Hence two write operations from two different transactions or a write from T_1 and a read from T_2 are not allowed.
 - A data item can be modified only under Exclusive lock.
 - To modify a data item T_1 will do:
 - *Exclusive lock (X), then Write (X) and finally Unlock (X).*

...CON'T

- When these locks are applied, then a transaction must behave in a special way.
- This special behavior of a transaction is referred to as *well-formed*.
- ***Well-formed***: A transaction is well- formed if it does not lock a locked data item and it does not try to unlock an unlocked data item.

Locking - Basic Rules

- If transaction has a shared lock on an item, it can read but not update the item.
- If a transaction has an exclusive lock on an item, it can both read and update the item.
- Reads cannot conflict, so more than one transaction can hold shared locks simultaneously on same item.
- Exclusive lock gives transaction exclusive access to that item.
- Some systems allow transaction to upgrade a shared lock to an exclusive lock, or vice-versa.

...CON'T

Examples: T_1 and T_2 are two transactions.

- They are executed under locking as follows.
- T_1 locks A in exclusive mode.
- When T_2 wants to lock A, it finds it locked by T_1 so T_2 waits for Unlock on A by T_1 .
- When A is released then T_2 locks A and begins execution.
- Suppose a lock on a data item is applied, the data item is processed and it is unlocked immediately after reading/writing is completed as follows. Initial values of A = 10 and B = 20.

...CON'T

Serial Execution of T1 and then T2	
T1	T2
Lock (A)	
read (A) {A = 10}	
A := A + 100	
write (A) (A = 110)	
Unlock (A)	
Lock (B)	
read (B) {B = 20}	
B := B + 10	
write (B) {B = 30}	
Unlock (B)	
	Lock (B)
	read (B) {B = 30}
	B := B * 5
	write (B) {B = 150}
	Unlock (B)
	Lock (A)
	Read (A) {A = 110}
	A := A + 20
	Write (A) {A = 130}
	Unlock (A)
Final Result: A=130 B=150	

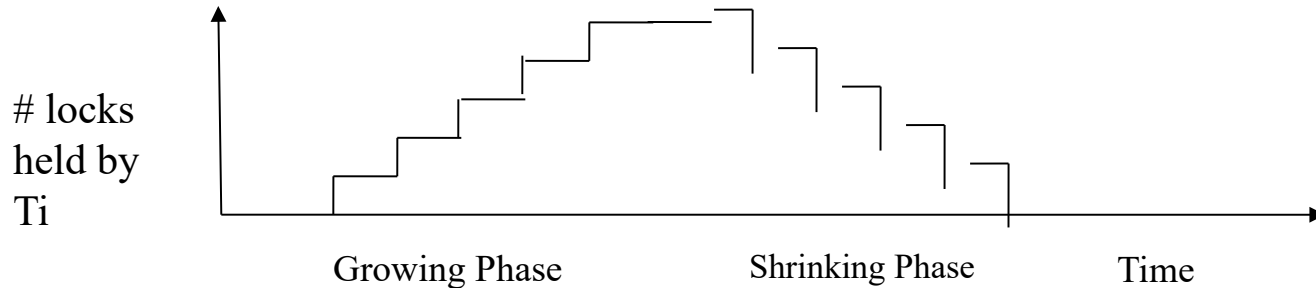
Concurrent Execution of T1 and T2	
T1	T2
Lock (A)	
read (A) {A = 10}	
A := A + 100	
write (A) (A = 110)	
Unlock (A)	
	Lock (B)
	read (B) {B = 20}
	B := B * 5
	write (B) {B = 100}
	Unlock (B)
Lock (B)	
read (B) {B = 100}	
B := B + 10	
write (B) {B = 110}	
Unlock (B)	
	Lock (A)
	Read (A) {A = 110}
	A := A + 20
	Write (A) {A = 130}
	Unlock (A)
Final Result: A=130 B=110	

...CON'T

- The final result of the two transactions using the two types of transaction execution (serial and concurrent) is not the same.
- This indicates that the above method of locking and unlocking is **not correct**.
- *This is because although such kind of locking and unlocking data items increases the concurrency of execution it violates the **isolation** and **atomicity** of transactions.*
- *Immediate unlocking is not reliable.*
- Thus, to preserve consistency we have to use another approach to locking, **two-phase locking** scheme.

Two-Phase Locking (2PL)

- A transaction follows 2PL protocol if all locking operations precede the first unlock operation in the transaction.
- The 2PL protocol demands locking and unlocking of a transaction to have two phases.
 - **(Locking) Growing phase** - acquires all locks but cannot release any locks. Upgrading of shared lock to exclusive locks is done here if allowed.
 - **(Unlocking) Shrinking phase** - releases locks but cannot acquire any new locks. Down grading of exclusive locks to shared lock is done here if allowed.
- Hence the 2PL protocol allows avoiding the three problems of concurrent execution.



Guaranteeing Serializability by Two-Phase Locking

- If every transaction in a schedule follows the two-phase locking protocol, schedule guaranteed to be serializable
- Two-phase locking may limit the amount of concurrency that can occur in a schedule
- Some serializable schedules will be prohibited by two-phase locking protocol

Variations of Two-Phase Locking

- Basic 2PL
 - Technique described on previous slides
- Conservative (static) 2PL
 - Requires a transaction to lock all the items it accesses before the transaction begins
 - Predeclare read-set and write-set
 - Deadlock-free protocol
- Strict 2PL
 - Transaction does not release exclusive locks until after it commits or aborts

Variations of Two-Phase Locking (cont'd.)

- Rigorous 2PL
 - Transaction does not release any locks until after it commits or aborts
- Concurrency control subsystem responsible for generating read_lock and write_lock requests
- Locking generally considered to have high overhead

Locking methods: problems

Deadlock: A deadlock that may result when two (or more) transactions are each waiting for locks held by the other to be released.

Deadlock - possible solutions

- **Only one way to break deadlock:**
 - **abort one or more of the transactions in the deadlock.**
- Deadlock should be transparent to user, so DBMS should restart transaction(s).
- Two general techniques for handling deadlock:
 - Deadlock prevention.
 - Deadlock detection and recovery.

...CON'T

i. Deadlock prevention protocol: two possibilities

➤ *The conservative two-phase locking*

- A transaction locks all data items it refers to before it begins execution.
- This way of locking prevents deadlock since a transaction never waits for a data item.
- **Limitation** : It restricts concurrency

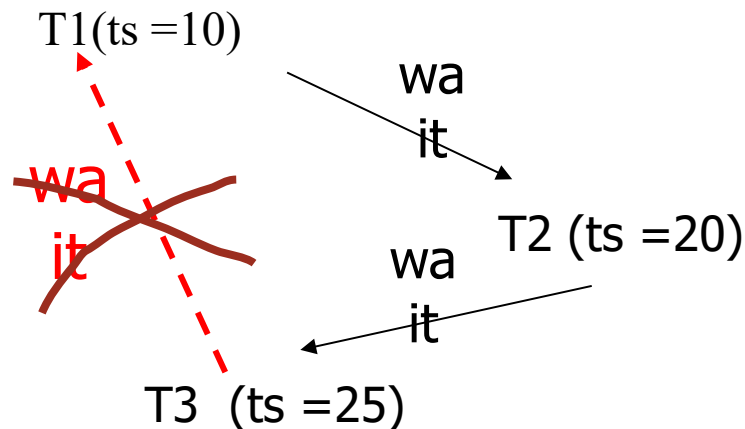
➤ *Transaction Timestamp($TS(T)$)*

- *We can prevent deadlocks by giving each transaction a priority and ensuring that lower priority transactions are not allowed to wait for higher priority transactions (or vice versa).*
- *One way to assign priorities is to give each transaction a **timestamp** when it starts up.*
- *it is a unique identifier given to each transaction based on time in which it is started. i.e if T_1 starts before T_2 , $TS(T_1) < TS(T_2)$*
- *The lower the timestamp, the higher the transaction's priority, that is, the oldest transaction has the highest priority.*
- *If a transaction T_i requests a lock and transaction T_j holds a conflicting lock, the lock manager can use one of the following two policies: Wait-die & Wound-wait*

...CON'T

- Wait-die

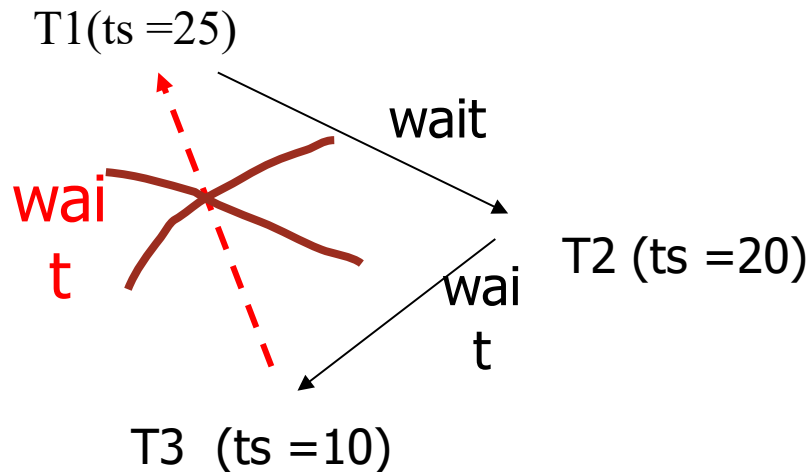
- If T_i has higher priority, it is allowed to wait; otherwise it is aborted.
- An older transaction is allowed to wait on a younger transaction.
- A younger transaction requesting an item held by an older transaction is aborted.
- If $TS(T_i) < TS(T_j)$, then (T_i older than T_j) T_i is allowed to wait.
- Otherwise (T_i younger than T_j) Abort T_i (T_i dies) and restart it later with the same timestamp.



...CON'T

- Wound-wait

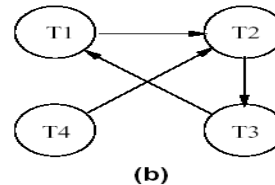
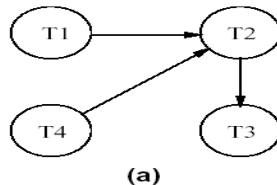
- The opposite of wait-die
- If T_i has higher priority, abort T_j ; otherwise T_i waits.
- A younger transaction is allowed to wait on all older one
- An older transaction requesting an item held by a younger transaction preempts the younger transaction by aborting it.
- If $TS(T_i) < TS(T_j)$, then (T_i older than T_j), Abort T_j (T_i wounds T_j) and restart T_j later with the same timestamp
 - Otherwise (T_i younger than T_j) T_i is allowed to wait



...CON'T

ii. Deadlock Detection and resolution

- In this approach, deadlocks are allowed to happen
- The scheduler maintains a wait-for-graph for detecting cycle.
- When a chain like: T_i waits for T_j waits for T_k waits for T_i or T_j occurs, then this creates a cycle.
- When the system is in the state of deadlock, some of the transaction should be aborted by selected (victim) and rolled-back
- This can be done by aborting those transaction: **that have made the least work, the one with the lowest locks, and that have the least # of abortion and so on.**
- Example



Time out

- The deadlock detection could be done using the technique of TIMEOUT.
- Every transaction will be given a time to wait in case of deadlock.
- If a transaction waits for the predefined period of time in idle mode, the DBMS will assume that deadlock occurred and it will abort and restart the transaction.
- WFG (the Wait For Graph)
 - Nodes represent transactions
 - A directed edge $T_i \rightarrow T_j$ is drawn if T_i is waiting to lock an item already locked by T_j .
 - Dead lock exists if there is a cycle in the graph.

...CON'T

- If every transaction in a schedule follows 2PL, schedule is serializable.
- However, problems can occur with interpretation when locks can be released.
- Suppose T_i aborts, but T_j has read a data item written by T_i
- Then T_j must abort; if T_j had been allowed to commit earlier, the schedule is not recoverable.
- Further, any transaction that has read a data item written by T_j must also abort
- This can lead to cascading rollback...
- In order to avoid such problems, it is recommended to leave the release of locks until the end of the transaction.
- ***A schedule is recoverable if for each pair of transactions T_i and T_j , if T_j reads an item previously written by T_i , then the commit operation of T_i must precede that of T_j .***

Time-stamping Method

- **Timestamp:** a unique identifier created by DBMS that indicates relative starting time of a transaction.
- Can be generated by:
 - ❑ Using system clock at the time transaction started, or
 - ❑ Incrementing a logical counter every time a new transaction starts.
- Time-stamping → a concurrency control protocol that orders transactions in such a way that older transactions, transactions with **smaller time stamps**, get **priority** in the event of **conflict**.

Starvation

- Occurs if a transaction cannot proceed for an indefinite period of time while other transactions continue normally
- Solution: first-come-first-served queue

...CON'T

- Transactions ordered globally based on their timestamp so that older transactions, transactions with earlier timestamps, get priority in the event of conflict.
- Conflict is resolved by rolling back and restarting transaction.
- Since there is no need to use lock there will be **No Deadlock**.
- In timestamp ordering, the schedule is equivalent to the particular serial order that corresponds to the order of the transaction timestamps.
- To implement this scheme, every transaction will be given a timestamp which is a unique identifier of a transaction.

...CON'T

- If T_i came to processing prior to T_j then TS of T_j will be larger than TS of T_i .
- Again each data item will have a timestamp for Read and Write.
 - **WTS(A)** which denotes the largest timestamp of any transaction that successfully executed **Write(A)**
 - **RTS(A)** which denotes the largest timestamp of any transaction that successfully executed **Read(A)**
- These timestamps are updated whenever a new Read (A) or Write (A) instruction is executed.
- Read/write proceeds only if last update on that data item was carried out by an **older transaction**.
- Otherwise, transaction requesting read/write is restarted and given a new timestamp.
- The timestamp ordering protocol ensures that any conflicting read and write operations are executed in the timestamp order.
- The protocol manages concurrent execution such that the timestamps determine the serializability order.

...CON'T

- ***Rules for permitting execution of operations in Time-stamping Method***
 - Suppose that Transaction T_i issues Read(A)
 - If $TS(T_i) < WTS(A)$: this implies that T_i needs to read a value of A which was already overwritten. Hence the **read** operation must be **rejected** and T_i is **rolled back**.
 - If $TS(T_i) \geq WTS(A)$: then the read is executed and $RTS(A)$ is set to the maximum of $RTS(A)$ and $TS(T_i)$.
 - Suppose that Transaction T_i issues Write (A)
 - If $TS(T_i) < RTS(A)$: then this implies that the value of A that T_i is producing was previously needed and it was assumed that it would never be produced. Hence, the Write operation must be **rejected** and T_i is **rolled back**.
 - If $TS(T_i) < WTS(A)$: then this implies that T_i is attempting to Write an object value of A. hence, this write operation can be **ignored**.
 - Otherwise the Write operation is executed and $WTS(A)$ is set to the maximum of $WTS(A)$ or $TS(T_i)$.
 - A transaction that is rolled back due to **conflict** will be **restarted** and be given a **new timestamp**.

Optimistic Technique

- Locking, assigning and checking timestamp values may be unnecessary for some transactions
- It assumes that conflict is **rare**.
- When transaction reaches the level of executing commit, a check is performed to determine whether conflict has occurred.
- If there is a **conflict**, transaction is **rolled back** and **restarted**.
- Based on assumption that conflict is rare and more efficient to let transactions proceed **without delays** to ensure serializability.
- At commit, check is made to determine whether conflict has occurred.
- If there is a conflict, transaction must be rolled back and restarted.
- Potentially allows greater concurrency than traditional protocols.
- Three phases:
 - Read
 - Validation
 - Write

...CON'T

1) Optimistic Techniques - Read Phase

- Extends from **start** until immediately before **commit**.
- Transaction reads values from database and stores them in **local variables**. Updates are applied to a **local copy of the data**.

2) Optimistic Techniques - Validation Phase

- Follows the **read phase**.
- For read-only transaction, checks that data read are still current values. If **no interference**, transaction is committed, else **aborted** and **restarted**.
- For update transaction, checks transaction leaves database in a consistent state, with serializability maintained.

3) Optimistic Techniques - Write Phase

- Follows successful **validation phase** for **update transactions**.
- Updates made to local copy are applied to the database.

Granularity of data items

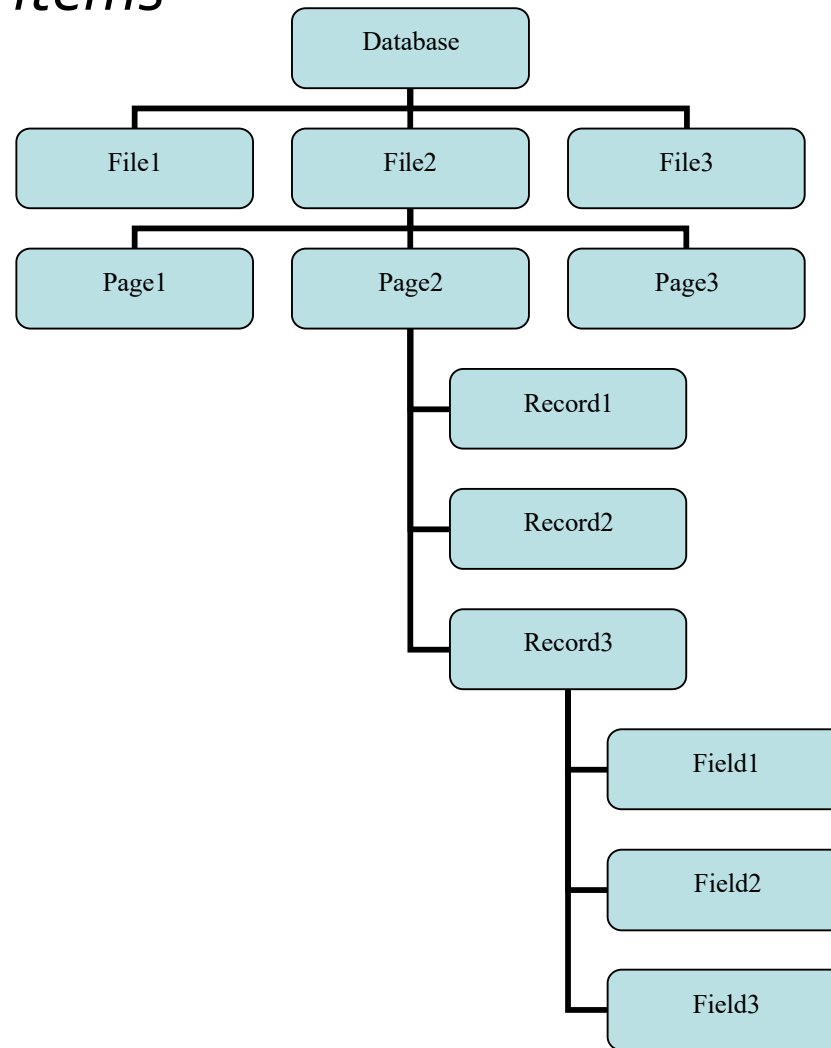
- **Granularity** is the size of the data items chosen as the unit of protection by a **concurrency control protocol**.
- See figure bellow.
- It could be:
 - The entire database
 - A file
 - A page (a section of physical disk in which relations are stored) (sometimes also called a **block**)
 - A record
 - A field value of a record
- The granularity has **effect on the performance of the system**.
- As locking will prevent access to the data, the size of the data required to be locked will **prevent** other transactions from having access.
- If the entire database is locked, then consistency will be highly maintained but **less performance** of the system will be witnessed.

...CON'T

- Is a single data item is locked; consistency maybe at risk but concurrent processing and performance will be enhanced.
- Thus, as one go from the entire database to a single value, performance and concurrent processing will be enhanced but consistency will be at risk and needs good concurrency control mechanism and strategy.

...CON'T

- *Granularity of data items*



END